Notes Management System

Project Overview

I Have Made This project as a **Full-Stack Notes Management System** developed as part of the Trainee Developer Assignment.
The application allows users to **create, view, edit, delete, and search notes efficiently** with a clean and responsive UI.

Core Task :

I didn't directly use the given code. It had multiple logical and syntax issues. I fixed those issues conceptually and implemented a cleaner version using MongoDB and proper APIs.

1. **Understood the logic**
2. **Fixed the bugs**
3. **Rebuilt it using MongoDB**

Although I identified logical errors, type mismatches, validation issues, and improved API correctness and reliability.

Objectives Achieved :

The project fulfills all required objectives:

- Bug fixing from given code (Core Task)
- Backend API development
- Frontend UI development
- Full-stack integration with database

Tech Stack

Frontend:

- React (Vite)
- Tailwind CSS
- Axios
- React Router

Backend:

- Node.js
- Express.js

Database:

- MongoDB (Mongoose)

Features Implemented

Notes Management

- Create multiple notes
- View all notes
- View single note details
- Edit/update notes
- Delete notes (with confirmation)

Search Functionality

- Search notes by:
    - Title
    - Content

Sorting

- Notes sorted by **latest updated/created**

Validation

- Title cannot be empty
- Proper error messages displayed
- Backend validation implemented

Authentication (Extra Feature)

- User Registration
- User Login
- Session handled using localStorage
- Notes linked to specific users

UI/UX Features

- Responsive design
- Premium modern UI (Tailwind + gradients)
- Loading states
- Empty states
- Clean dashboard layout

- Professional navigation

Backend API Endpoints

Method	Endpoint	Description
POST	/notes	Create note
GET	/notes	Get all notes
GET	/notes/:id	Get single note
PUT	/notes/:id	Update note
DELETE	/notes/:id	Delete note

Database Design

Note Schema:

- title
- content
- userId
- createdAt
- updatedAt

Core Task (Bug Fixing)

The provided buggy code had multiple issues such as:

- Incorrect variable usage
- Logical errors (assignment vs comparison)
- Missing validations
- Incorrect API responses
- Wrong data handling

Improvements Made:

- Fixed all syntax & logical bugs
- Improved validation logic
- Corrected API structure
- Enhanced reliability and readability

Additional Improvements (Beyond Assignment)

- Implemented authentication system
- Linked notes to specific users
- Designed premium UI
- Added reusable components
- Improved user experience

Project Structure

```
Frontend/
  ├── components/
  ├── pages/
  ├── services/
  ├── App.jsx

Backend/

  ├── controller.js
  ├── models/
  ├── routes/
  ├── server.js
```

How to Run Project

Backend:

```
cd backend
npm install
node server.js
```

Frontend:

```
cd frontend
npm install
npm run dev
```

Evaluation Criteria Covered

Problem Solving
Code Quality
API Design
Database Integration
Validation

UI/UX
Search Functionality
Responsiveness
Effort & Learning

Conclusion

This project demonstrates the ability to:

- Build a full-stack application
- Design REST APIs
- Integrate frontend with backend
- Manage database operations
- Create user-friendly interfaces